

Pré-Processamento de Imagens

O pré-Processamento de imagens tem como objetivo tratar e padronizar as imagens, pois num mundo cheio de dados sem padrão universal faz com que cada organização lide com tamanho de imagens diferentes, escala de cores diferentes e tudo mais, um dos exemplos que eu posso trazer é como analista de dados, no atual emprego lidamos com diversas plataformas e cada um com sua formatação própria, nos dando o trabalho de tratar cada um para trazer no padrão da base da empresa, nos forçando a criar automações e tudo mais, nas imagens não é diferente, pois, mesmo não sendo dados tabulares ainda, eles serão para tratar com o modelo, e para isso precisamos fazer com que todos os dados sigam o mesmo padrão, porque assim que o modelo for treinado, ele vai esperar aquele modelo específico para fazer a predição e qualquer alteração pode resultar em erros, tanto no sistema quanto por parte do modelo ali treinado.

Trazendo bibliotecas, resolvi falar sobre o Pytorch, um framework que lida com Redes Neurais e por sua vez é bem legível, fácil de entender e aplicar e junto com esse framework ele traz o torchvision que importa o transforms, que oferece transformações já prontas. Como o Resize que traz um redimensionamento (muito importante para o processamento de imagens, afim de padronizar o tamanho de todas as imagens e reduzir para o modelo não consumir memórias desnecessárias), temos o corte central, por meio do CenterCrop que corta o centro da imagem, na qual ele entende que é onde está o conteúdo principal para a rede. E como estamos lidando com inteligência artificial, temos que ter em mente que pelo menos ainda os modelos não são treinados por outra forma a não ser com números e matemática por traz dos panos, para transformar as imagens para números na qual a IA vai entender e aprender, utilizamos o toTensor, na qual se parece muito com numpy array, ele transforma a imagem em arrays fazendo com que seja aceito nos modelos de redes neurais. Além disso temos a normalização, Data Augmentation, pipelines e muito mais para realizar o pré-processamento.

Abaixo irei trazer exemplos de como realizar um pré-processamento de imagens:

```
"""
```

```
A Compose serve para aplicar a pipeline, fazendo com que possamos aplicar  
vários processamentos de uma vez só nas imagens ou conjunto de imagens
```

```
"""
```

```
transformacao = transforms.Compose([
```

```
    """
```

```
    Caso haja imagens coloridas no conjunto de imagens  
    o Grayscale ficará encarregado de transforma ela em escala de cinza  
    para que o modelo não precise se esforçar muito em aprender com 3 escalas de  
    cores
```

```
    """
```

```
    transforms.Grayscale(num_output_channels=1),
```

```
    # como mencionado no resumo, o Resize irá redimensionar as imagens
```

```
    # abaixo será de 128x128 padronizando todas as imagens e diminuindo cada uma delas
```

```
    transforms.Resize((128, 128)),
```

```
    # Transforma todas as imagens em tensores
```

```
    transforms.ToTensor(),
```

```
    """ como não expliquei antes, vale a explicação agora. A normalização ajusta  
    os valores dos dados de entrada para que fiquem numa escala padronizada,  
    geralmente com média 0 e desvio padrão 1. Isso evita que valores maiores  
    influenciem mais o treinamento do que deveriam. Os valores passam a ficar  
    entre -1 e 1, o que ajuda o modelo a treinar de forma mais estável e eficiente
```

```
    """
```

```
    transforms.Normalize(mean=[0.5], std=[0.5])
```

```
])
```

Segmentação de imagens

A segmentação de imagens é uma técnica usada para dividir uma imagem em partes significativas. Diferente de uma classificação, que diz o que está na imagem e de detecção de objetos, a segmentação meio que marca cada pixel da imagem como pertencente a uma classe específica. Temos alguns tipos principais de segmentação, a semântica, de instância e panóptica.

Semântica: cada pixel recebe um rótulo de classe, mas não diferencia diferentes instâncias, um exemplo é a segmentação de um carro, a semântica não vai segmentar como carro 1 e carro 2, vai segmentar como carro por exemplo.

Instância: diferencia instâncias da mesma classe, aqui o carro vai receber um rótulo de carro 1, carro 2, etc.

Panóptica: combina os dois tipos acima.

Um exemplo disso utilizando o Pytorch (já que utilizei no exemplo anterior) seria usando uma rede pré-treinada, para não ter que criar uma do zero, existe uma rede chamada DeeplabV3 com um backbone ResNet50 do torchvision. No exemplo abaixo não trarei o pré-processamento, para não ficar com repetições de código no resumo.

```
import torchvision

# Instancia o modelo pre treinado do torchvision
# esse modelo é pré treinado com a semântica
model = torchvision.models.segmentation.deeplabv3_resnet50(pretrained=True)
model.eval()

"""
esse no_grad desativa o cálculo do gradiente durante a execução
é útil quando estamos fazendo apenas a predição e não o treinamento,
tendo em vista que o modelo já está treinado ne
"""

with torch.no_grad():
    # digamos que o input_tensor seja uma imagem transformada em tensor
    output = model(input_tensor)['out'][0]
    # Isso pega o índice da classe com maior valor para cada pixel
    predictions = output.argmax(0).byte().cpu().numpy()
```

Detecção/Classificação de Imagens

Detecção e classificação de imagens focam em ensinar uma IA a identificar o conteúdo de uma imagem. A diferença principal é que na classificação, um rótulo é atribuído à imagem toda (ex: "é um carro"), enquanto na detecção, o sistema localiza e classifica o objeto na imagem simultaneamente (ex: "o carro está aqui").

Essa tecnologia é comum no dia a dia, como em aplicativos que organizam fotos por categorias. Na área médica, que é o foco do meu exemplo, a classificação pode identificar a presença de doenças em exames, como um tumor cerebral.

Para a IA aprender, ela precisa de muitos exemplos. Usamos milhares de imagens já rotuladas para que o modelo reconheça padrões visuais repetitivos entre as classes. Para o caso de tumores, ele aprende os padrões presentes em cérebros com e sem a condição.

Utilizei a biblioteca PyTorch para criar e treinar as redes neurais. O modelo implementado é uma Rede Neural Convolutiva, ideal para essa tarefa por sua capacidade de captar características visuais importantes. O torchvision também foi usado para conjuntos de dados e transformações.

No exemplo prático, com ajuda de cursos que faço a respeito, desenvolvi um classificador binário para identificar a presença de tumor em imagens de ressonância magnética. A estrutura do modelo inclui:

- **Camadas convolucionais:** Identificam padrões locais.
- **Camadas de ativação (ReLU):** Introduzem não-linearidade.
- **Batch Normalization:** Acelera o treinamento.
- **MaxPooling:** Reduz o tamanho e destaca padrões relevantes.
- **Camadas lineares:** Tomam a decisão final.
- **Dropout:** Previne overfitting.

O processo de aprendizado usa a função de perda **BCEWithLogitsLoss**, adequada para problemas binários, e o otimizador **Adam**, que ajusta os pesos do modelo.

```

class Classificador(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)
        self.conv2 = nn.Conv2d(32, 32, kernel_size=3)
        self.activation = nn.ReLU()
        self.bnorm = nn.BatchNorm2d(32)
        self.pool = nn.MaxPool2d(kernel_size=2)
        self.flatten = nn.Flatten()
        self.linear1 = nn.Linear(32 * 30 * 30, 128)
        self.linear2 = nn.Linear(128, 128)
        self.output = nn.Linear(128, 1)
        self.dropout = nn.Dropout(0.2)

    def forward(self, X):
        X = self.pool(self.bnorm(self.activation(self.conv1(X))))
        X = self.pool(self.bnorm(self.activation(self.conv2(X))))
        X = self.flatten(X)
        X = self.dropout(self.activation(self.linear1(X)))
        X = self.dropout(self.activation(self.linear2(X)))
        X = self.output(X)
        return X

# Após montar a rede, basta instanciá-la, definir o otimizador e a função de perda, e por fim executar o
loop de treinamento:
model = Classificador().to(device)
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

for epoch in range(4):
    model.train()
    for imgs, labels in train_loader:
        imgs, labels = imgs.to(device), labels.to(device).float().view(-1, 1)

        optimizer.zero_grad()
        outputs = model(imgs).squeeze(1)
        loss = criterion(outputs, labels.squeeze(1))
        loss.backward()
        optimizer.step()

```